

Find a Group of Three Numbers Within Distance d

Company: Google

Round: Offline

Year: 2022

Difficulty: Medium–Hard

Topic: Arrays, Sorting, Sliding Window, Data Structures

Source: <https://www.designa.in/5722/google-interview-experiences-sde-set-4>

Problem Description

You are given a distance d and a **stream of floating-point numbers**. The numbers arrive **one at a time**.

For every new number received, check whether there is a **group of three numbers** such that the distance between every pair of numbers is at most d .

If such a group of three numbers is found:

1. Remove those three numbers from the collection.
2. Return the group.

Input Format

- The first line contains a floating-point number d , representing the maximum allowed distance between the numbers.
- The second line contains an integer N , representing the number of numbers in the stream.
- The next N lines contain one floating-point number at a time, representing the numbers arriving in the stream.

Output Format

- If a group of **three numbers** is found where the difference between the smallest and largest number is at most d , print those three numbers.
- The three numbers should then be **removed from the collection**.
- If no such group exists, print:

Sample Input

```
2.0
6
1.0
5.0
2.5
3.0
10.0
4.0
```

Sample Output

```
1.0 2.5 3.0
```

Explanation

We are given a distance d and numbers that arrive **one at a time**.

For every new number, we need to check whether there are **three numbers** in our collection such that the distance between the smallest and largest number is at most d .

Example

Suppose:

$$d = 2.0$$

The numbers arrive as:

1.0

5.0

2.5

3.0

10.0

4.0

Consider these three numbers:

1.0, 2.5, 3.0

The smallest number is:

1.0

and the largest is:

3.0

Their difference is:

$$3.0 - 1.0 = 2.0$$

Since:

$$2.0 \leq d$$

these three numbers form a valid group.

Therefore, we return:

1.0 2.5 3.0

and **remove these three numbers from the collection.**

Why checking the smallest and largest is enough

If three numbers are sorted as:

$$a \leq b \leq c$$

we only need to check:

$$c - a \leq d$$

If this condition is true, then b is automatically within distance d of both a and c .

Approach to Solve This Problem:

1. Set the given distance as d .
2. Receive each number from the stream one at a time.
3. Insert the number into a sorted collection.

4. Check possible groups of 3 nearby numbers.
5. For three sorted numbers:
 $a \leq b \leq c$
we only need to check:
 $c - a \leq d$
6. If the condition is satisfied:
 - Return these three numbers.
 - Remove all three numbers from the collection.
7. If no valid group exists, continue waiting for the next number.

Java Solution:

```
1 import java.util.*;
2 public class Main {
3     static TreeMap<Double, Integer> numbers = new TreeMap<>();
4     static List<Double> addNumber(double x, double d) {
5         numbers.put(x, numbers.getOrDefault(x, 0) + 1);
6         if (numbers.size() >= 2) {
7             Double first = numbers.firstKey();
8             while (first != null) {
9                 Double second = numbers.higherKey(first);
10                if (second == null) {
11                    break;
12                }
13                Double third = numbers.higherKey(second);
14                if (third == null) {
15                    break;
16                }
17                if (third - first <= d) {
18                    removeNumber(first);
19                    removeNumber(second);
20                    removeNumber(third);
21                    return Arrays.asList(first, second, third);
22                }
23                first = second;
24            }
25        }
26        return null;
27    }
28 }
```

```

27     }
28     static void removeNumber(double x) {
29         int count = numbers.get(x);
30         if (count == 1) {
31             numbers.remove(x);
32         } else {
33             numbers.put(x, count - 1);
34         }
35     }
36     public static void main(String[] args) {
37         Scanner sc = new Scanner(System.in);
38         double d = sc.nextDouble();
39         int n = sc.nextInt();
40         for (int i = 0; i < n; i++) {
41             double x = sc.nextDouble();
42             List<Double> result = addNumber(x, d);
43             if (result != null) {
44                 System.out.println(
45                     result.get(0) + " " +
46                     result.get(1) + " " +
47                     result.get(2)
48                 );
49             }
50         }
51         sc.close();
52     }
53 }

```

Solution:

The solution is to maintain all incoming numbers in **sorted order** and look for any three numbers that satisfy the distance condition.

Suppose three numbers in sorted order are:

$$a \leq b \leq c$$

For these three numbers to form a valid group, we only need to check:

$$c - a \leq d$$

If this condition is satisfied:

1. The three numbers form a valid group.
2. Return these three numbers.
3. Remove them from the collection.
4. Continue processing the remaining incoming numbers.

Example

Given:

$$d = 2$$

and the numbers:

1.0, 5.0, 2.5, 3.0

When 3.0 arrives, the sorted numbers include:

1.0 2.5 3.0 5.0

Take:

1.0, 2.5, 3.0

Check:

$$3.0 - 1.0 = 2.0$$

Since:

$$2.0 \leq d$$

we found a valid group.

So:

Return $\rightarrow$ 1.0 2.5 3.0

and remove them from the collection.

Why This Works

Because the numbers are sorted, the **smallest and largest numbers determine the maximum distance** between any two numbers in the group.

Therefore, checking:

$$\text{largest} - \text{smallest} \leq d$$

is sufficient.

Complexity Analysis:

Time Complexity: $O(N^2 \log N)$

- Each of the N incoming numbers is inserted into the TreeMap in $O(\log N)$ time.
- For every new number, the code may scan the sorted collection to find a valid group of three.
- In the worst case, this scan can take $O(N)$ time.
- Therefore, the overall worst-case complexity is approximately:
 $O(N \times N \times \log N) = O(N^2 \log N)$

Space Complexity: $O(N)$

- The TreeMap stores the numbers that have not yet been removed.
- In the worst case, all N numbers may remain in the collection.

Therefore:

$$\text{Space Complexity} = O(N)$$